Atividade Prática — Desenvolvimento Orientado por Especificações (Spec-Driven Development)

Engenharia de Software — Sistemas de Informação

1. Contexto

Em projetos de software, uma das dificuldades mais comuns não é escrever código, mas **transformar uma necessidade de negócio em um software que realmente atenda ao que foi solicitado**.

Antes de implementar uma funcionalidade, é necessário compreender:

- qual problema está sendo resolvido;
- quem utilizará o sistema;
- quais comportamentos são esperados;
- quais regras de negócio precisam ser respeitadas;
- o que está dentro e fora do escopo;
- como saberemos se a funcionalidade está correta.

Nesta atividade, você irá experimentar uma abordagem conhecida como **Spec-Driven Development (SDD)**, ou **Desenvolvimento Orientado por Especificações**.

A ideia central é simples:

> **A implementação deve ser orientada por uma especificação clara e verificável.**

Em vez de começar diretamente escrevendo código, você seguirá um processo semelhante a:

Problema → Especificação → Esclarecimento → Planejamento → Tarefas → Implementação → Testes → Validação

Você poderá utilizar ferramentas de Inteligência Artificial para auxiliar em algumas etapas, mas a responsabilidade pelas decisões do projeto continuará sendo sua.

---

2. Situação-problema

Imagine que você faça parte de uma equipe de desenvolvimento contratada para construir um sistema interno para uma empresa.

A empresa possui atualmente um processo informal para registrar problemas de seus funcionários:

- solicitações são enviadas por e-mail ou mensagens;
- algumas solicitações acabam esquecidas;
- não existe um histórico organizado;
- não é possível acompanhar facilmente o andamento de uma solicitação;
- gestores têm dificuldade para saber quais problemas estão pendentes.

A empresa deseja inicialmente um **MVP de um Sistema de Chamados de Suporte (Help Desk)**.

O sistema deverá permitir que funcionários registrem solicitações de suporte e acompanhem sua situação.

A equipe de desenvolvimento recebeu apenas essa descrição inicial.

Seu trabalho será transformar essa ideia em uma especificação suficientemente clara para orientar a implementação.

---

3. O que você deverá desenvolver

Você deverá produzir um pequeno sistema Web de chamados.

O sistema deverá permitir, pelo menos:

- cadastrar um chamado;
- visualizar chamados cadastrados;
- consultar os dados de um chamado;
- alterar o status de um chamado;
- registrar informações básicas sobre o atendimento.

O projeto poderá ser implementado utilizando tecnologias que você conheça ou queira experimentar.

Por exemplo:

- HTML, CSS e JavaScript;
- JavaScript + Node.js;
- Python + Flask/Django;
- PHP;
- outro stack Web adequado ao projeto.

Não existe uma única solução tecnológica obrigatória.

Entretanto, a tecnologia escolhida deve ser compatível com o tamanho do projeto e permitir que você consiga implementar e demonstrar o MVP.

---

4. Objetivos da atividade

Ao final da atividade, você deverá ser capaz de:

1. transformar uma necessidade de negócio em uma especificação de software;
2. identificar ambiguidades e informações ausentes em requisitos;
3. definir critérios de aceitação verificáveis;
4. decompor um projeto em funcionalidades menores;
5. elaborar um plano de implementação a partir de uma especificação;
6. utilizar uma ferramenta de IA como apoio ao processo de desenvolvimento;
7. implementar funcionalidades respeitando uma especificação previamente definida;

8. elaborar e executar testes baseados nos critérios de aceitação;
9. avaliar criticamente uma implementação produzida com auxílio de IA;
10. compreender na prática a relação entre requisitos, planejamento, implementação e testes.

---

5. Ferramentas

A atividade será iniciada em sala e poderá ser concluída em casa.

Durante a aula, utilize as ferramentas disponíveis no laboratório.

Em casa, você poderá escolher livremente as ferramentas que considerar mais adequadas.

Por exemplo:

- VS Code ou outro editor/IDE;
- Git e GitHub;
- uma ferramenta de IA generativa;
- Node.js;
- Python;
- Java;
- PHP;
- banco de dados;
- frameworks ou bibliotecas.

A escolha das ferramentas faz parte da atividade.

Entretanto, **não escolha uma tecnologia apenas porque ela é mais sofisticada**. Considere se ela é adequada ao problema e se você conseguirá utilizá-la de forma produtiva.

---

6. Organização do projeto

Crie uma pasta para o projeto.

Uma possível organização inicial é:

```
help-desk/
│
├── project.md
├── roadmap.md
│
├── features/
│   ├── 01-criar-chamado.md
│   ├── 02-listar-chamados.md
│   ├── 03-consultar-chamado.md
│   └── 04-atualizar-chamado.md
```

```
└── src/
```

Os arquivos `.md` serão utilizados para registrar as especificações do projeto.

Você pode utilizar o Markdown diretamente no VS Code ou em outro editor.

---

7. Etapa 1 — Defina a missão do projeto

Comece criando o arquivo:

```
project.md
```

Nesse arquivo, descreva o projeto em termos de negócio.

Utilize inicialmente esta estrutura:

```
# Projeto: Sistema de Chamados

## Missão

Descreva o problema que o sistema pretende resolver.

## Público-alvo

Quem utilizará o sistema?

## Problema

Qual problema existe atualmente?

## Objetivos

O que o sistema deverá possibilitar?

## Funcionalidades principais

Liste as principais funcionalidades previstas.

## Fora do escopo

O que o sistema NÃO fará neste projeto?

## Restrições

Existem limitações técnicas, de prazo ou de infraestrutura?
```

```
## Stack tecnológica

### Front-end

### Back-end

### Banco de dados

### Outras tecnologias
```

Não tente detalhar todas as funcionalidades neste momento.

O objetivo desta etapa é estabelecer **a visão do produto**.

---

8. Etapa 2 — Defina o MVP

MVP significa **Minimum Viable Product**, ou Produto Mínimo Viável.

É a primeira versão do sistema que possui funcionalidades suficientes para entregar valor e permitir sua avaliação.

Uma armadilha comum em projetos acadêmicos é tentar implementar tudo de uma vez.

Por exemplo, o sistema poderia futuramente possuir:

- autenticação;
- diferentes perfis de usuário;
- notificações por e-mail;
- anexos;
- comentários;
- prioridades;
- categorias;
- SLA;
- dashboard;
- relatórios;
- integração com e-mail;
- integração com Microsoft Teams;
- inteligência artificial para classificação dos chamados.

Nada disso precisa fazer parte do primeiro MVP.

Comece pequeno.

Uma possibilidade seria definir como MVP:

> Um usuário consegue registrar um chamado, visualizar os chamados existentes, consultar seus detalhes e alterar seu status.

As funcionalidades adicionais podem ser registradas como **post-MVP**.

9. Etapa 3 — Crie o roadmap

Crie:

```
roadmap.md
```

O roadmap representa uma visão geral da evolução do projeto.

Por exemplo:

```
# Roadmap

## MVP

### Fase 1 — Estrutura inicial
- Criar estrutura do projeto
- Configurar ambiente
- Criar página inicial

### Fase 2 — Cadastro de chamados
- Criar formulário
- Validar dados
- Armazenar chamado

### Fase 3 — Consulta
- Listar chamados
- Visualizar detalhes

### Fase 4 — Atendimento
- Alterar status
- Registrar informações do atendimento

## Pós-MVP

- Autenticação
- Perfis de usuário
- Prioridade
- Categorias
- Anexos
- Notificações
- Dashboard
```

Não considere o roadmap como um plano imutável.

Durante o desenvolvimento, você poderá descobrir que uma funcionalidade precisa ser dividida, modificada ou removida.

Isso também faz parte do processo de engenharia.

---

10. Etapa 4 — Use IA para revisar a definição do projeto

Agora você poderá utilizar uma ferramenta de IA generativa.

Envie para a ferramenta:

```
project.md
roadmap.md
```

Utilize um prompt semelhante a:

```
Estou desenvolvendo um sistema de chamados para uma disciplina de Engenharia de
Software.

Analise os arquivos project.md e roadmap.md.

Não escreva código.

Avalie:

1. Se a missão do projeto está clara.
2. Se existem ambiguidades importantes.
3. Se existem requisitos implícitos que deveriam ser explicitados.
4. Se o MVP está suficientemente delimitado.
5. Se alguma funcionalidade deveria ser dividida em partes menores.
6. Se existem dependências entre funcionalidades.
7. Se existem funcionalidades desnecessárias para o MVP.

Apresente perguntas que eu deveria responder antes de começar a implementação.

Não tome decisões pelo projeto. Quando houver uma decisão de negócio ou requisito
ambíguo, apresente as alternativas e explique o impacto de cada uma.
```

Leia criticamente a resposta.

Não aceite automaticamente as sugestões da IA.

Você é responsável por decidir quais mudanças fazem sentido.

---

11. Etapa 5 — Especifique uma funcionalidade

Agora vamos sair da visão geral do projeto e trabalhar com uma funcionalidade específica.

Comece por:

```
features/01-criar-chamado.md
```

Utilize a seguinte estrutura:

```
# Criar chamado

## Objetivo

Qual problema esta funcionalidade resolve?

## Atores

Quem pode utilizar esta funcionalidade?

## User Story

Como [tipo de usuário],
quero [ação],
para [benefício].

## Dados de entrada

Quais informações precisam ser fornecidas?

## Regras de negócio

Quais regras devem ser respeitadas?

## Comportamento esperado

O que o sistema deve fazer?

## Validações

Quais dados devem ser validados?

## Critérios de aceitação

Como podemos verificar objetivamente se a funcionalidade está correta?

## Casos de exceção

O que deve acontecer quando algo inesperado ocorre?

## Fora do escopo

O que não será implementado nesta funcionalidade?
```

```
## Status

Especificação inicial
```

---

12. Requisitos não são a mesma coisa que critérios de aceitação

Essa distinção é importante.

Um **requisito** descreve algo que o sistema deve fazer.

Por exemplo:

> O usuário deve informar o título do chamado.

Um **critério de aceitação** descreve uma condição objetiva que permite verificar se o requisito foi atendido.

Por exemplo:

> Ao tentar enviar um chamado sem informar o título, o sistema deve impedir o cadastro e apresentar uma mensagem indicando que o campo é obrigatório.

O critério de aceitação deve permitir responder:

> **Passou ou não passou?**

Isso será importante posteriormente nos testes.

---

13. Exemplo de especificação

Uma especificação parcial poderia ser:

```
# Criar chamado

## Objetivo

Permitir que um usuário registre uma nova solicitação de suporte.

## Atores

Funcionário.

## User Story

Como funcionário,
quero registrar um problema de suporte,
```

```
para que a equipe responsável possa realizar o atendimento.

## Dados de entrada

- Título
- Descrição
- Categoria

## Regras de negócio

- O título é obrigatório.
- A descrição é obrigatória.
- Todo chamado deve possuir um status.
- Um novo chamado deve iniciar com status "Aberto".

## Critérios de aceitação

1. Deve ser possível preencher título, descrição e categoria.
2. Ao enviar um formulário válido, um novo chamado deve ser criado.
3. Um chamado recém-criado deve possuir status "Aberto".
4. O sistema deve impedir o cadastro quando o título não for informado.
5. O sistema deve impedir o cadastro quando a descrição não for informada.

## Fora do escopo

- Upload de arquivos.
- Notificação por e-mail.
- Autenticação.

## Status

Especificação inicial
```

Observe que ainda podem existir perguntas.

Por exemplo:

- Qual é o tamanho máximo do título?
- Qual é o tamanho máximo da descrição?
- Categoria é obrigatória?
- Quais categorias existem?
- O usuário pode editar um chamado depois de criá-lo?
- O que acontece se ocorrer um erro ao salvar?

Essas perguntas fazem parte do processo.

---

14. Etapa 6 — Faça a sessão de esclarecimento

Envie a especificação para a IA.

Use, por exemplo:

```
Analise a especificação da funcionalidade apresentada neste arquivo.

Não escreva código.

Procure especificamente:

- ambiguidades;
- requisitos incompletos;
- regras de negócio não definidas;
- casos de exceção;
- critérios de aceitação que não sejam verificáveis;
- informações que possam gerar interpretações diferentes durante a implementação.

Faça perguntas para esclarecer esses pontos.

Não invente requisitos. Quando houver mais de uma possibilidade válida, apresente
as alternativas.
```

Agora você deverá responder às perguntas relevantes.

Atualize o arquivo `.md`.

Essa etapa é importante porque uma especificação não deve ser considerada definitiva apenas porque foi escrita.

Ela deve ser **compreendida e suficientemente esclarecida**.

---

15. Etapa 7 — Elabore o plano de implementação

Somente depois que a especificação estiver suficientemente definida, solicite um plano.

Utilize um prompt como:

```
Com base no project.md, roadmap.md e na especificação da funcionalidade
apresentada, produza um plano de implementação.

Não escreva código.

Divida a implementação em tarefas pequenas e verificáveis.

Para cada tarefa, informe:

- objetivo;
- arquivos ou componentes envolvidos;
- dependências;
- resultado esperado;
```

```
- como verificar se a tarefa foi concluída.

Respeite rigorosamente a especificação.

Não adicione funcionalidades que não estejam previstas.
```

O resultado deverá ser um **plano**, e não código.

---

16. Etapa 8 — Transforme o plano em tarefas

Um plano pode ser dividido em tarefas como:

```
# Plano – Criar chamado

## Tarefa 1
Criar a estrutura do formulário.

## Tarefa 2
Adicionar os campos título, descrição e categoria.

## Tarefa 3
Implementar validação dos campos obrigatórios.

## Tarefa 4
Implementar criação do chamado.

## Tarefa 5
Definir o status inicial como "Aberto".

## Tarefa 6
Apresentar confirmação após o cadastro.

## Tarefa 7
Implementar tratamento de erro.
```

Quanto menor e mais verificável for uma tarefa, mais fácil será identificar problemas durante a implementação.

---

17. Etapa 9 — Implemente a funcionalidade

Agora você poderá utilizar a IA para auxiliar na implementação.

Forneça:

- `project.md`;
- `roadmap.md`;

- especificação da funcionalidade;
- plano de implementação.

Um prompt possível:

```
Implemente a funcionalidade descrita na especificação.

Utilize project.md, roadmap.md e a especificação como fonte de verdade.

Respeite:

- requisitos;
- regras de negócio;
- critérios de aceitação;
- stack tecnológica definida.

Não adicione funcionalidades que não foram solicitadas.

Explique quais arquivos precisam ser criados ou alterados e apresente a
implementação.

Se existir alguma informação necessária que não esteja definida na especificação,
pare e faça uma pergunta em vez de assumir uma decisão de negócio.
```

A IA passa a atuar como **apoio à implementação**, e não como responsável pelo projeto.

---

18. Etapa 10 — Teste a funcionalidade

Agora volte aos critérios de aceitação.

Crie uma tabela de validação:

Critério	Resultado esperado	Resultado obtido	Situação
Criar chamado válido	Chamado criado		
Título vazio	Cadastro impedido		
Descrição vazia	Cadastro impedido		
Novo chamado	Status "Aberto"		

Execute os testes.

Não considere a funcionalidade concluída apenas porque:

> "O código executou sem apresentar erro."

O objetivo é verificar se **o software atende à especificação**.

---

19. O que fazer quando um teste falhar?

Essa situação é especialmente importante.

Imagine que a especificação diga:

> Um novo chamado deve iniciar com status "Aberto".

Mas o sistema cria o chamado com status "Em atendimento".

Não altere imediatamente a especificação para fazer o teste passar.

Primeiro investigue.

Pode existir:

1. um erro na implementação;
2. uma especificação incompleta;
3. uma regra de negócio equivocada;
4. uma interpretação diferente da especificação.

Utilize a IA para auxiliar na investigação:

```
O critério de aceitação abaixo não foi atendido.

Especificação:
[cole o critério]

Comportamento esperado:
[descreva]

Comportamento observado:
[descreva]

Analise possíveis causas.

Não altere a especificação automaticamente.

Primeiro determine se o problema está na implementação ou se existe alguma
inconsistência na especificação.
```

Esse procedimento ajuda a evitar uma prática comum no desenvolvimento assistido por IA:

> alterar o requisito apenas para que o código pareça estar correto.

---

20. Etapa 11 — Feche a funcionalidade

Uma funcionalidade somente deverá ser considerada concluída quando:

- [ ] a necessidade foi definida;
- [ ] a especificação foi escrita;
- [ ] as ambiguidades relevantes foram esclarecidas;
- [ ] as regras de negócio foram definidas;
- [ ] os critérios de aceitação foram definidos;
- [ ] o plano foi elaborado;
- [ ] as tarefas foram executadas;
- [ ] a implementação foi realizada;
- [ ] os testes foram executados;
- [ ] os critérios de aceitação foram atendidos;
- [ ] eventuais problemas foram corrigidos;
- [ ] a especificação foi atualizada quando necessário.

Altere então o status da funcionalidade para:

```
## Status

Concluída
```

---

21. Etapa 12 — Atualize o roadmap

Depois de concluir uma funcionalidade, atualize:

```
roadmap.md
```

Por exemplo:

```
### Fase 2 — Cadastro de chamados

- [x] Criar formulário
- [x] Validar dados
- [x] Armazenar chamado
```

Essa atualização permite acompanhar a evolução do projeto.

---

22. Repita o processo

Agora aplique o mesmo processo às demais funcionalidades.

Por exemplo:

Funcionalidade 01

Criar chamado.

Funcionalidade 02

Listar chamados.

Funcionalidade 03

Consultar detalhes do chamado.

Funcionalidade 04

Atualizar status.

Para cada uma:

Especificar → Esclarecer → Planejar → Implementar → Testar → Validar → Encerrar

Não desenvolva todas as funcionalidades simultaneamente.

A proposta é experimentar o desenvolvimento **incremental**.

---

23. Evolução do sistema

Depois de concluir o MVP, escolha uma funcionalidade adicional.

Por exemplo:

- prioridade do chamado;
- categoria;
- autenticação;
- comentários;
- anexos;
- filtro por status;
- busca;
- dashboard.

Antes de implementá-la, repita novamente o processo.

Isso permite observar uma característica importante da Engenharia de Software:

> **O software continua evoluindo depois que a primeira versão foi entregue.**

---

24. IA como ferramenta de Engenharia de Software

Durante a atividade, você poderá utilizar IA para:

- analisar requisitos;
- encontrar ambiguidades;
- sugerir casos de exceção;
- auxiliar na elaboração de critérios de aceitação;
- produzir planos de implementação;
- decompor tarefas;
- gerar código;
- explicar código;
- sugerir testes;
- investigar erros;
- revisar uma implementação.

Entretanto, a IA não deve ser tratada como autoridade sobre o projeto.

Uma resposta produzida por IA pode:

- interpretar incorretamente um requisito;
- inventar informações;
- adicionar funcionalidades;
- escolher uma solução tecnológica inadequada;
- produzir código incorreto;
- ignorar uma regra de negócio.

Portanto:

> **A IA pode ajudar a produzir o software, mas não substitui a responsabilidade de engenharia.**

---

25. Um princípio importante: a especificação como fonte de verdade

Durante a atividade, utilize a seguinte regra:

> **O código deve estar de acordo com a especificação.**

Não faça o contrário:

> "O código está funcionando, então vamos dizer que era isso que o requisito queria."

Quando houver conflito entre código e especificação, investigue o motivo.

Essa prática aproxima o exercício de situações reais de desenvolvimento, nas quais requisitos, implementação e testes precisam permanecer alinhados.

---

26. Entrega final

Ao final da atividade, o projeto deverá conter pelo menos:

```
help-desk/
│
├── project.md
├── roadmap.md
│
├── features/
│   ├── 01-criar-chamado.md
│   ├── 02-listar-chamados.md
│   ├── 03-consultar-chamado.md
│   └── 04-atualizar-chamado.md
│
├── src/
│
└── README.md
```

O `README.md` deverá apresentar:

- descrição do sistema;
- objetivo;
- funcionalidades implementadas;
- tecnologias utilizadas;
- instruções para executar o projeto;
- limitações conhecidas;
- funcionalidades futuras.

---

27. Relato sobre o uso de IA

Inclua no README ou em um arquivo separado:

```
AI-USAGE.md
```

Descreva:

- qual ferramenta de IA foi utilizada;
- em quais etapas ela foi utilizada;
- exemplos de tarefas realizadas com auxílio da IA;
- quais sugestões da IA foram rejeitadas;
- quais problemas foram encontrados nas respostas;
- como você verificou o código produzido;
- quais decisões permaneceram sob responsabilidade da equipe.

O objetivo não é avaliar "quanto código foi produzido pela IA".

O objetivo é compreender **como a IA foi incorporada ao processo de desenvolvimento**.

---

28. Questões para reflexão

Ao finalizar o projeto, responda:

1.

Em que momentos a especificação evitou interpretações diferentes sobre o comportamento do sistema?

2.

Quais ambiguidades foram identificadas antes da implementação?

3.

Houve alguma situação em que a IA sugeriu uma solução que não estava de acordo com a especificação?

4.

O que aconteceu quando uma implementação não passou em um critério de aceitação?

5.

Qual foi a relação entre requisitos, critérios de aceitação e testes?

6.

O trabalho de especificação tornou a implementação mais fácil ou mais difícil? Por quê?

7.

Quais partes do processo poderiam ser automatizadas com ferramentas de IA?

8.

Quais decisões ainda precisam necessariamente de participação humana?

9.

O que você faria diferente se esse sistema fosse um produto real utilizado por uma empresa?

---

29. O processo completo

Ao final, você terá experimentado um fluxo semelhante a:

```
NECESSIDADE DE NEGÓCIO
        ↓
MISSÃO DO PROJETO
        ↓
MVP
```

```
        ↓
ROADMAP
        ↓
FUNCIONALIDADE
        ↓
ESPECIFICAÇÃO
        ↓
ESCLARECIMENTO
        ↓
PLANO
        ↓
TAREFAS
        ↓
IMPLEMENTAÇÃO
        ↓
TESTES
        ↓
VALIDAÇÃO
        ↓
FUNCIONALIDADE CONCLUÍDA
        ↓
PRÓXIMA FUNCIONALIDADE
        ↓
MVP
        ↓
EVOLUÇÃO
```

Esse é o principal conceito que você deve levar desta atividade:

> **Desenvolver software não significa apenas escrever código. Significa transformar uma necessidade em uma solução, tomando decisões, registrando o que deve ser construído, verificando o resultado e evoluindo o sistema de forma controlada.**

O Spec-Driven Development é uma forma de organizar esse processo colocando a **especificação como elemento central entre a necessidade e a implementação**.